

# Notetaking & Organization

---

Thorough notetaking is critical during any assessment. Our notes, accompanied by tool and log output, are the raw inputs to our draft report, which is typically the only portion of our assessment that our client sees. Even though we usually keep our notes for ourselves, we must keep things organized and develop a repeatable process to save time and facilitate the reporting process. Detailed notes are also a must in the event of a network issue or client question (i.e., did you scan X host on Y day?), so being overly verbose in our notetaking never hurts. Everyone will have their own style they are comfortable with and should work with their preferred tools and organizational structure to ensure the best possible results. In this module, we will cover the minimum elements that, from our professional experience, should be noted down during an assessment (or even while working through a large module, playing a box on HTB, or taking an exam) to save time and energy come reporting time or as a reference guide down the road. If you're part of a larger team where someone may have to cover a client meeting for you, clear and consistent notes are essential to ensure your teammate can speak confidently and accurately about what activities were and were not performed.

---

## Notetaking Sample Structure

There is no universal solution or structure for notetaking as each project and tester is different. The structure below is what we have found to be helpful but should be adapted to your personal workflow, project type, and the specific circumstances you encountered during your project. For example, some of these categories may not be applicable for an application-focused assessment and may even warrant additional categories not listed here.

- **Attack Path** - An outline of the entire path if you gain a foothold during an external penetration test or compromise one or more hosts (or the AD domain) during an internal penetration test. Outline the path as closely as possible using screenshots and command output will make it easier to paste into the report later and only need to worry about formatting.
- **Credentials** - A centralized place to keep your compromised credentials and secrets as you go along.
- **Findings** - We recommend creating a subfolder for each finding and then writing our narrative and saving it in the folder along with any evidence (screenshots, command output). It is also worth keeping a section in your notetaking tool for recording findings information to help organize them for the report.
- **Vulnerability Scan Research** - A section to take notes on things you've researched and tried with your vulnerability scans (so you don't end up redoing work you already did).
- **Service Enumeration Research** - A section to take notes on which services you've investigated, failed exploitation attempts, promising vulnerabilities/misconfigurations, etc.
- **Web Application Research** - A section to note down interesting web applications found through various methods, such as subdomain brute-forcing. It's always good to perform thorough subdomain enumeration externally, scan for common web ports on internal assessments, and run a tool such as Aquatone or EyeWitness to screenshot all applications. As you review the screenshot report, note down applications of interest, common/default credential pairs you tried, etc.
- **AD Enumeration Research** - A section for showing, step-by-step, what Active Directory enumeration you've already performed. Note down any areas of interest you need to run down later in the assessment.
- **OSINT** - A section to keep track of interesting information you've collected via OSINT, if applicable to the engagement.
- **Administrative Information** - Some people may find it helpful to have a centralized location to store contact information for other project stakeholders like Project Managers (PMs) or client Points of Contact (POCs). unique objectives/flaas defined in

Other project stakeholders like Project Managers (PMs), or client forms of contact (POCs), unique objectives, huge defined in the Rules of Engagement (RoE), and other items that you find yourself often referencing throughout the project. It can also be used as a running to-do list. As ideas pop up for testing that you need to perform or want to try but don't have time for, be diligent about writing them down here so you can come back to them later.

- **Scoping Information** - Here, we can store information about in-scope IP addresses/CIDR ranges, web application URLs, and any credentials for web applications, VPN, or AD provided by the client. It could also include anything else pertinent to the scope of the assessment so we don't have to keep re-opening scope information and ensure that we don't stray from the scope of the assessment.
- **Activity Log** - High-level tracking of everything you did during the assessment for possible event correlation.
- **Payload Log** - Similar to the activity log, tracking the payloads you're using (and a file hash for anything uploaded and the upload location) in a client environment is critical. More on this later.

---

## Notetaking Tools

There are many tools available for notetaking, and the choice is very much personal preference. Here are some of the options available:

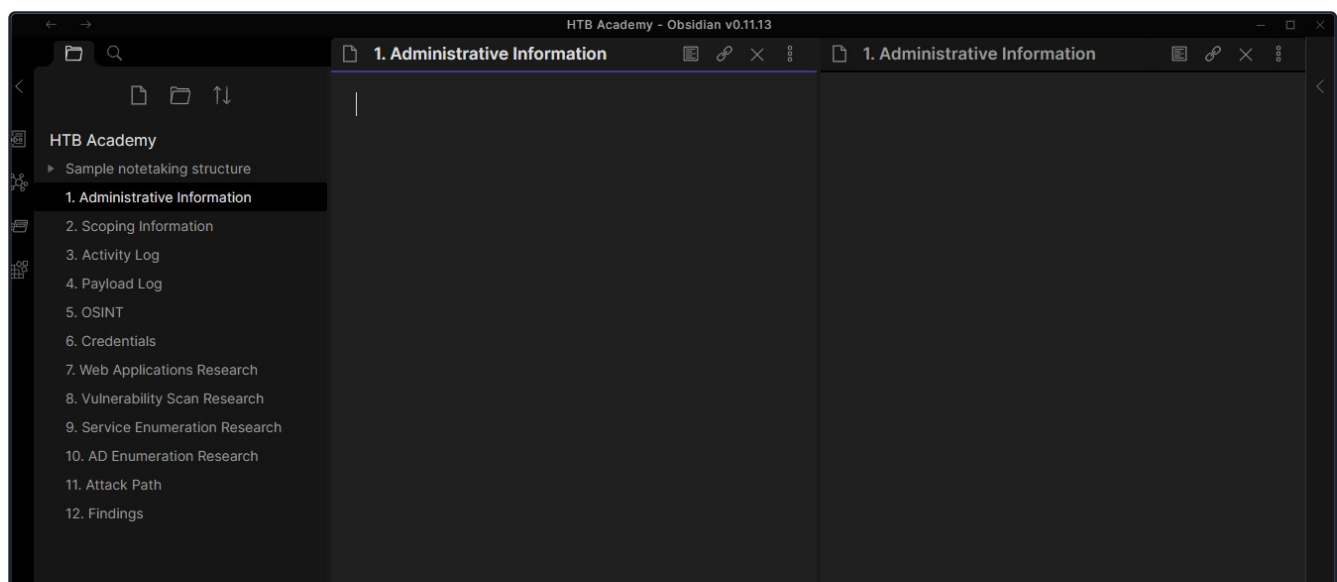
---

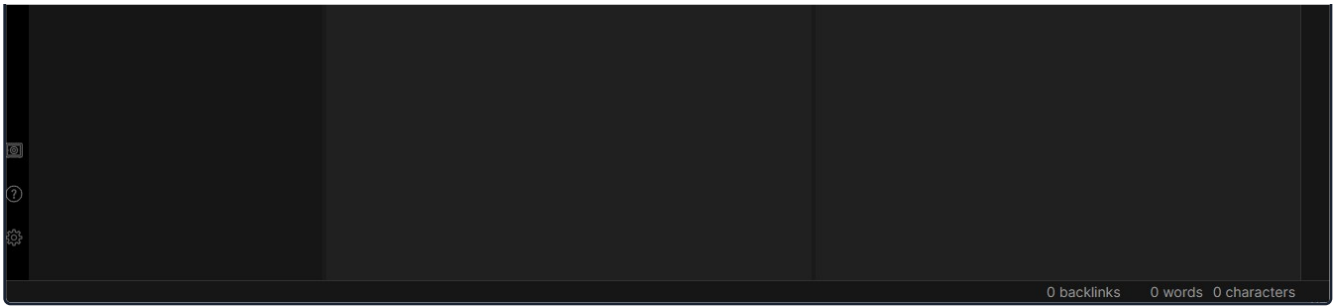
|                            |                                    |                                |
|----------------------------|------------------------------------|--------------------------------|
| <a href="#">CherryTree</a> | <a href="#">Visual Studio Code</a> | <a href="#">Evernote</a>       |
| <a href="#">Notion</a>     | <a href="#">GitBook</a>            | <a href="#">Sublime Text</a>   |
| <a href="#">Notepad++</a>  | <a href="#">OneNote</a>            | <a href="#">Outline</a>        |
| <a href="#">Obsidian</a>   | <a href="#">Cryptpad</a>           | <a href="#">Standard Notes</a> |

---

As a team, we've had many discussions about the pros and cons of various notetaking tools. One key factor is distinguishing between local and cloud solutions before choosing a tool. A cloud solution is likely acceptable for training courses, CTFs, labs, etc., but once we get into engagements and managing client data, we must be more careful with the solution we choose. Your company will likely have some sort of policy or contractual obligations around data storage, so it is best to consult with your manager or team lead on whether or not using a specific notetaking tool is permitted. **Obsidian** is an excellent solution for local storage, and **Outline** is great for the cloud but also has a [Self-hosted version](#). Both tools can be exported to Markdown and imported into any other tool that accepts this convenient format.

### Obsidian





Again, tools are personal preferences from person to person. Requirements typically vary from company to company, so experiment with different options and find one that you are comfortable with and practice with different setups and formats while working through Academy modules, HTB boxes, Pro Labs, and other pieces of training to get comfortable with your notetaking style while remaining as thorough as possible.

## Logging

It is essential that we log all scanning and attack attempts and keep raw tool output wherever possible. This will greatly help us come reporting time. Though our notes should be clear and extensive, we may miss something, and having our logs to fallback can help us when either adding more evidence to a report or responding to a client question.

## Exploitation Attempts

[Tmux logging](#) is an excellent choice for terminal logging, and we should absolutely be using [Tmux](#) along with logging as this will save every single thing that we type into a Tmux pane to a log file. It is also essential to keep track of exploitation attempts in case the client needs to correlate events later on (or in a situation where there are very few findings and they have questions about the work performed). It is supremely embarrassing if you cannot produce this information, and it can make you look inexperienced and unprofessional as a penetration tester. It can also be a good practice to keep track of things you tried during the assessment but did not work. This is especially useful for those instances in which we have little to no findings in your report. In this case, we can write up a narrative of the types of testing performed, so the reader can understand the kinds of things they are adequately protected against. We can set up Tmux logging on our system as follows:

First, clone the [Tmux Plugin Manager](#) repo to our home directory (in our case [/home/htb-student](#) or just `~`).

| Notetaking & Organization                                                                         |
|---------------------------------------------------------------------------------------------------|
| <pre>dbcyph0n@htb[/htb]\$ git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm</pre> |

Next, create a `.tmux.conf` file in the home directory.

| Notetaking & Organization                        |
|--------------------------------------------------|
| <pre>dbcyph0n@htb[/htb]\$ touch .tmux.conf</pre> |

The config file should have the following contents:

| Notetaking & Organization                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>dbcyph0n@htb[/htb]\$ cat .tmux.conf  # List of plugins  set -g @plugin 'tmux-plugins/tpm' set -g @plugin 'tmux-plugins/tmux-sensible' set -a @plugin 'tmux-plugins/tmux-loadline'</pre> |

```
# Initialize TMUX plugin manager (keep at bottom)
run '~/.tmux/plugins/tpm/tpm'
```

After creating this config file, we need to execute it in our current session, so the settings in the `.tmux.conf` file take effect. We can do this with the `source` command.

#### Notetaking & Organization

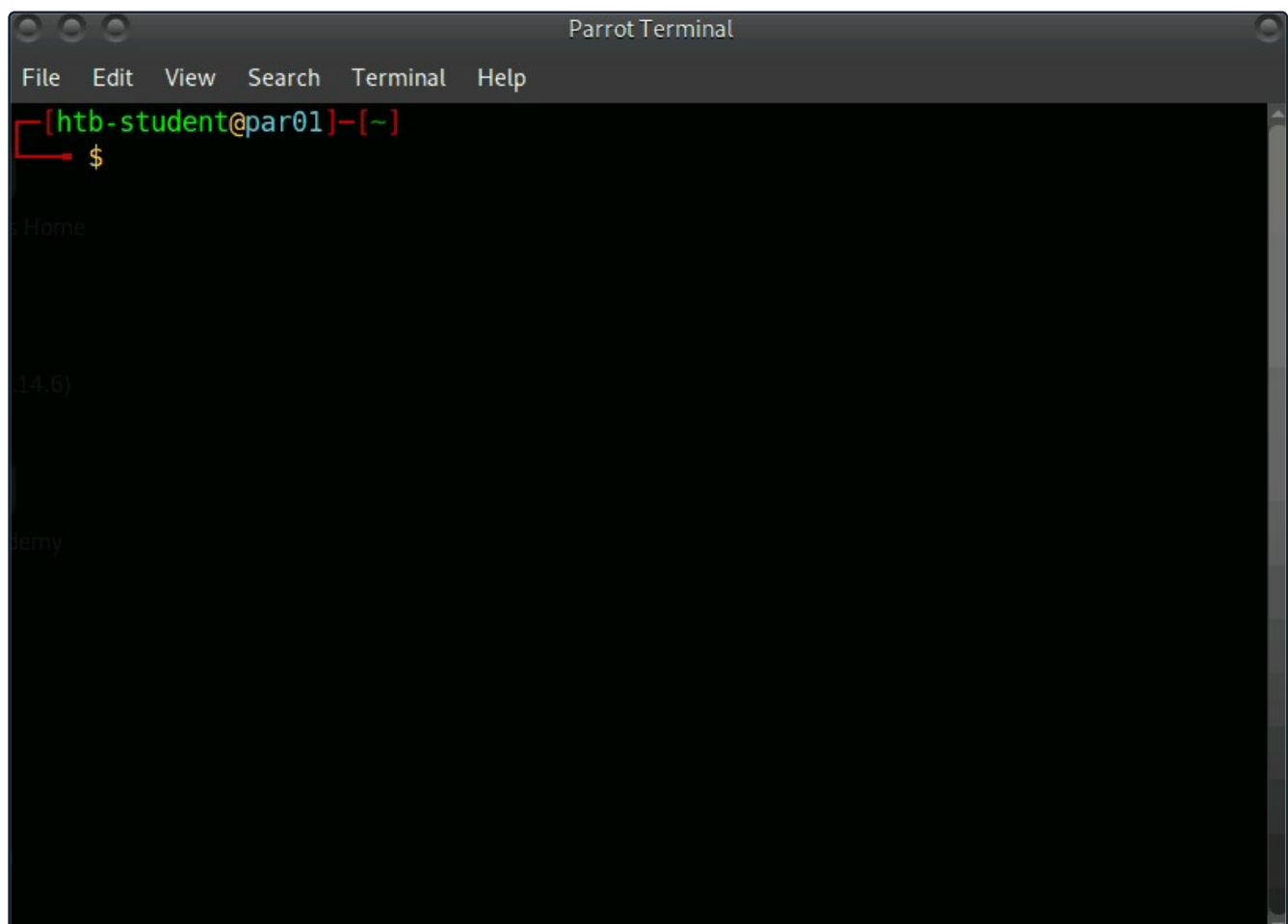
```
dbcypth0n@htb[/htb]$ tmux source ~/.tmux.conf
```

Next, we can start a new Tmux session (i.e., `tmux new -s setup`).

Once in the session, type `[Ctrl] + [B]` and then hit `[Shift] + [I]` (or `prefix + [Shift] + [I]` if you are not using the default prefix key), and the plugin will install (this could take around 5 seconds to complete).

Once the plugin is installed, start logging the current session (or pane) by typing `[Ctrl] + [B]` followed by `[Shift] + [P]` (`prefix + [Shift] + [P]`) to begin logging. If all went as planned, the bottom of the window will show that logging is enabled and the output file. To stop logging, repeat the `prefix + [Shift] + [P]` key combo or type `exit` to kill the session. Note that the log file will only be populated once you either stop logging or exit the Tmux session.

Once logging is complete, you can find all commands and output in the associated log file. See the demo below for a short visual on starting and stopping Tmux logging and viewing the results.



If we forget to enable Tmux logging and are deep into a project, we can perform retroactive logging by typing `[Ctrl] + [B]` and then hitting `[Alt] + [Shift] + [P]` (`prefix + [Alt] + [Shift] + [P]`), and the entire pane will be saved. The amount of saved data depends on the Tmux `history-limit` or the number of lines kept in the Tmux scrollback buffer. If this is left at the default value and

we try to perform retroactive logging, we will most likely lose data from earlier in the assessment. To safeguard against this situation, we can add the following lines to the `.tmux.conf` file (adjusting the number of lines as we please):

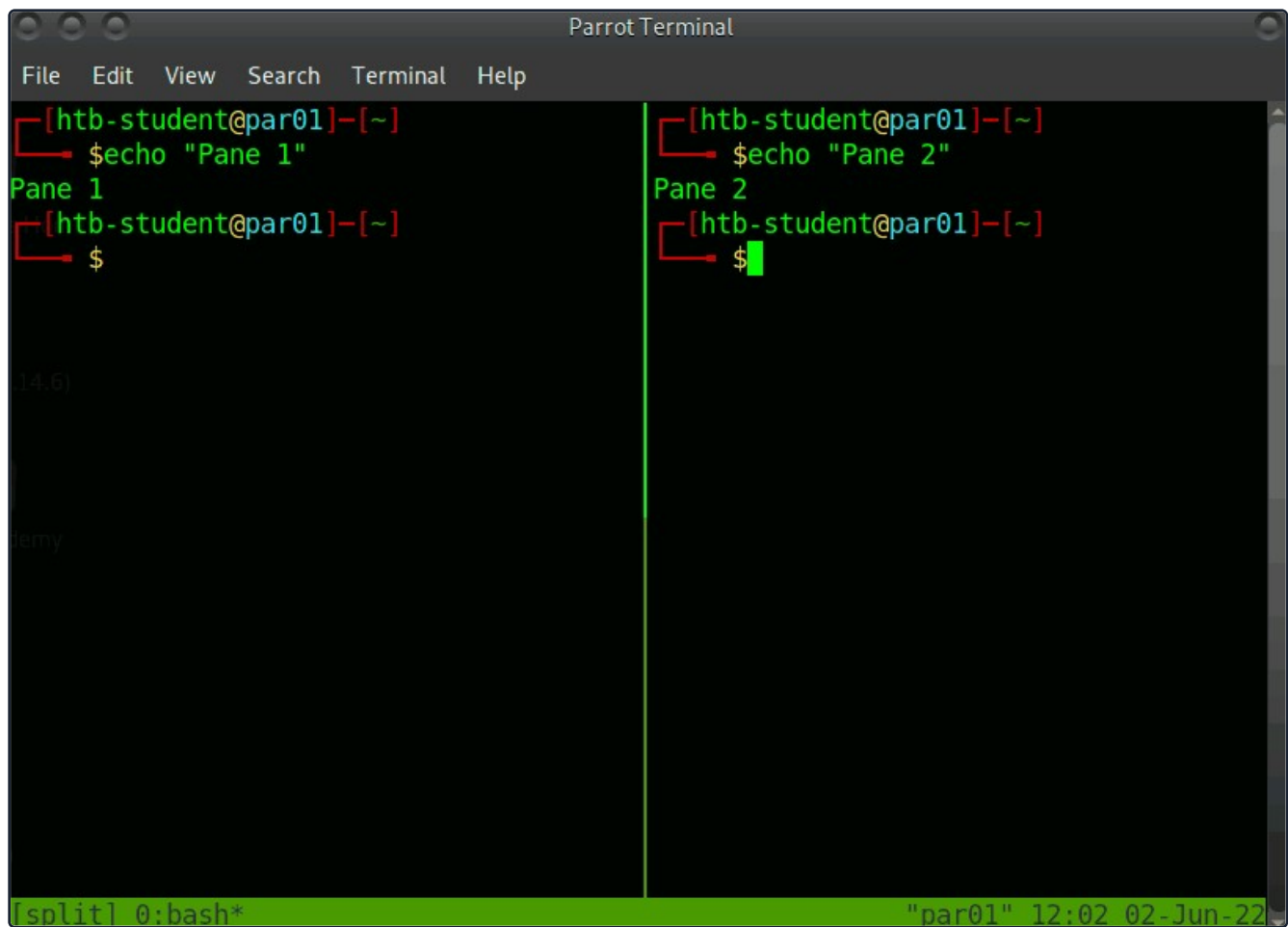
## Tmux.conf

### Notetaking & Organization

```
set -g history-limit 50000
```

Another handy trick is the ability to take a screen capture of the current Tmux window or an individual pane. Let's say we are working with a split window (2 panes), one with `Responder` and one with `ntlmrelayx.py`. If we attempt to copy/paste the output from one pane, we will grab data from the other pane along with it, which will look very messy and require cleanup. We can avoid this by taking a screen capture as follows: `[Ctrl] + [B]` followed by `[Alt] + [P]` (prefix + `[Alt] + [P]`). Let's see a quick demo.

Here we can see we're working with two panes. If we try to copy text from one pane, we'll grab text from the other pane, which would make a mess of the output. But, with Tmux logging enabled, we can take a capture of the pane and output it neatly to a file.



To recreate the above example first start a new tmux session: `tmux new -s sessionname`. Once in the session type `[Ctrl] + [B] + [Shift] + [%]` (prefix + `[Shift] + [%]`) to split the panes vertically (replace the `[%]` with `["]` to do a horizontal split). We can then move from pane to pane by typing `[Ctrl] + [B] + [0]` (prefix + `[0]`).

Finally, we can clear the pane history by typing `[Ctrl] + [B]` followed by `[Alt] + [C]` (prefix + `[Alt] + [C]`).

There are many other things we can do with Tmux, customizations we can do with Tmux logging (i.e. [changing the default logging path](#), changing key bindings, running multiple windows within sessions and panes within those windows, etc). It is worth reading up on all the capabilities that Tmux offers and finding out how the tool best fits your workflow. Finally, here are some additional plugins that we like:



- [tmux-sessionist](#) - Gives us the ability to manipulate Tmux sessions from within a session: switching to another session, creating a new named session, killing a session without detaching Tmux, promote the current pane to a new session, and more.
- [tmux-pain-control](#) - A plugin for controlling panes and providing more intuitive key bindings for moving around, resizing, and splitting panes.
- [tmux-resurrect](#) - This extremely handy plugin allows us to restore our Tmux environment after our host restarts. Some features include restoring all sessions, windows, panes, and their order, restoring running programs in a pane, restoring Vim sessions, and more.

Check out the complete [tmux plugins list](#) to see if others would fit nicely into your workflow. For more on Tmux, check out this excellent [video](#) by lppsec and this [cheat sheet](#) based on the video.

---

## Artifacts Left Behind

At a minimum, we should be tracking when a payload was used, which host it was used on, what file path it was placed in on the target, and whether it was cleaned up or needs to be cleaned up by the client. A file hash is also recommended for ease of searching on the client's part. It's best practice to provide this information even if we delete any web shells, payloads, or tools.

### Account Creation/System Modifications

If we create accounts or modify system settings, it should be evident that we need to track those things in case we cannot revert them once the assessment is complete. Some examples of this include:

- IP address of the host(s)/hostname(s) where the change was made
- Timestamp of the change
- Description of the change
- Location on the host(s) where the change was made
- Name of the application or service that was tampered with
- Name of the account (if you created one) and perhaps the password in case you are required to surrender it

It should go without saying, but as a professional and to prevent creating enemies out of the infrastructure team, you should get written approval from the client before making these types of system modifications or doing any sort of testing that might cause an issue with system stability or availability. This can typically be ironed out during the project kickoff call to determine the threshold beyond which the client is willing to tolerate without being notified.

---

## Evidence

No matter the assessment type, our client (typically) does not care about the cool exploit chains we pull off or how easily we "pwned" their network. Ultimately, they are paying for the report deliverable, which should clearly communicate the issues discovered and evidence that can be used for validation and reproduction. Without clear evidence, it can be challenging for internal security teams, sysadmins, devs, etc., to reproduce our work while working to implement a fix or even to understand the nature of the issue.

### What to Capture

As we know, each finding will need to have evidence. It may also be prudent to collect evidence of tests that were performed that were unsuccessful in case the client questions your thoroughness. If you're working on the command line, Tmux logs may be sufficient evidence to paste into the report as literal terminal output, but they can be horribly formatted. For this reason, capturing your terminal output for significant steps as you go along and tracking that separately alongside your findings is a good idea. For

everything else, screenshots should be taken.

## Storage

Much like with our notetaking structure, it's a good idea to come up with a framework for how we organize the data collected during an assessment. This may seem like overkill on smaller assessments, but if we're testing in a large environment and don't have a structured way to keep track of things, we're going to end up forgetting something, violating the rules of engagement, and probably doing things more than once which can be a huge time waster, especially during a time-boxed assessment. Below is a suggested baseline folder structure, but you may need to adapt it accordingly depending on the type of assessment you're performing or unique circumstances.

- Admin
  - Scope of Work (SoW) that you're working off of, your notes from the project kickoff meeting, status reports, vulnerability notifications, etc
- Deliverables
  - Folder for keeping your deliverables as you work through them. This will often be your report but can include other items such as supplemental spreadsheets and slide decks, depending on the specific client requirements.
- Evidence
  - Findings
    - We suggest creating a folder for each finding you plan to include in the report to keep your evidence for each finding in a container to make piecing the walkthrough together easier when you write the report.
  - Scans
    - Vulnerability scans
      - Export files from your vulnerability scanner (if applicable for the assessment type) for archiving.
    - Service Enumeration
      - Export files from tools you use to enumerate services in the target environment like Nmap, Masscan, Rumble, etc.
    - Web
      - Export files for tools such as ZAP or Burp state files, EyeWitness, Aquatone, etc.
    - AD Enumeration
      - JSON files from BloodHound, CSV files generated from PowerView or ADRecon, Ping Castle data, Snaffler log files, CrackMapExec logs, data from Impacket tools, etc.
  - Notes
    - A folder to keep your notes in.
  - OSINT
    - Any OSINT output from tools like Intelx and Maltego that doesn't fit well in your notes document.
  - Wireless
    - Optional if wireless testing is in scope, you can use this folder for output from wireless testing tools.
  - Logging output
    - Logging output from Tmux, Metasploit, and any other log output that does not fit the **Scan** subdirectories listed above.

- Misc Files
  - Web shells, payloads, custom scripts, and any other files generated during the assessment that are relevant to the project.
- Retest
  - This is an optional folder if you need to return after the original assessment and retest the previously discovered findings. You may want to replicate the folder structure you used during the initial assessment in this directory to keep your retest evidence separate from your original evidence.

It's a good idea to have scripts and tricks for setting up at the beginning of an assessment. We could take the following command to make our directories and subdirectories and adapt it further.

#### Notetaking & Organization

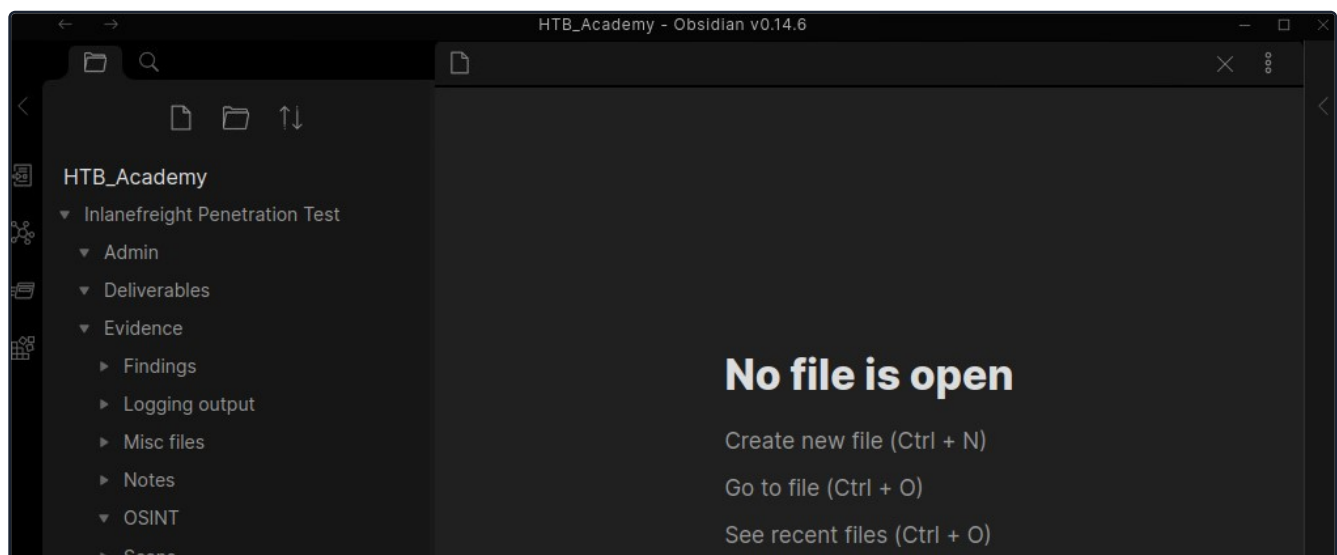
```
dbcyph0n@htb[/htb]$ mkdir -p ACME-IPT/{Admin,Deliverables,Evidence/{Findings,Scans/{Vuln,Service,Web,'AD Enume
```

#### Notetaking & Organization

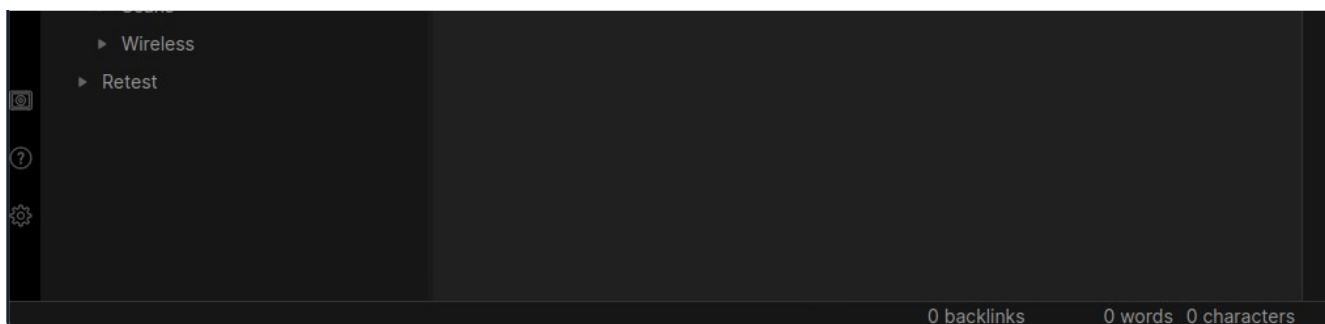
```
dbcyph0n@htb[/htb]$ tree ACME-IPT/

ACME-IPT/
├── Admin
├── Deliverables
├── Evidence
│   ├── Findings
│   ├── Logging output
│   ├── Misc Files
│   ├── Notes
│   ├── OSINT
│   ├── Scans
│   │   ├── AD Enumeration
│   │   ├── Service
│   │   ├── Vuln
│   │   └── Web
│   └── Wireless
└── Retest
```

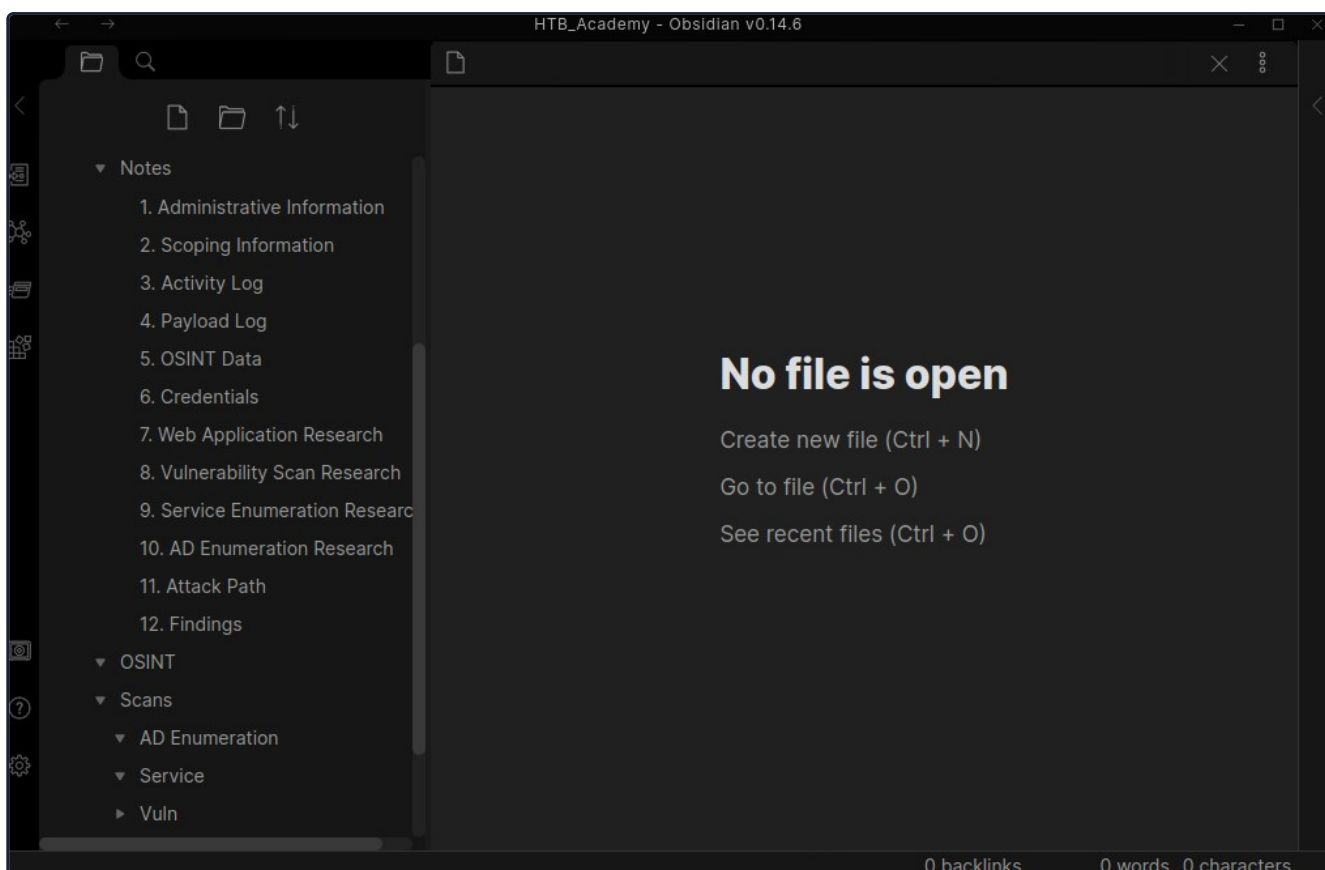
A nice feature of a tool such as Obsidian is that we can combine our folder structure and notetaking structure. This way, we can interact with the notes/folders directly from the command line or inside the Obsidian tool. Here we can see the general folder structure working through Obsidian.







Drilling down further, we can see the benefits of combining our notetaking and folder structure. During a real assessment, we may add additional pages/folders or remove some, a page and a folder for each finding, etc.

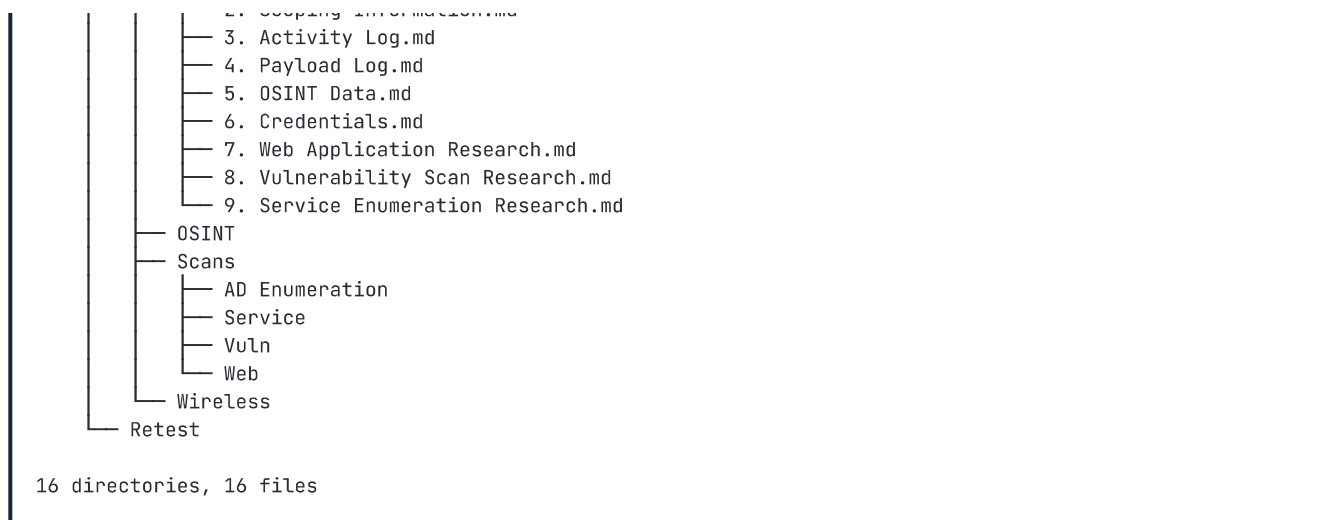


Taking a quick look at the directory structure, we can see each folder we created previously and some now populated with Obsidian Markdown pages.

```

Notetaking & Organization

dbcyph0n@htb[/htb]$ tree
.
├── Inlanefreight Penetration Test
│   ├── Admin
│   ├── Deliverables
│   ├── Evidence
│   │   ├── Findings
│   │   │   ├── H1 - Kerberoasting.md
│   │   │   ├── H2 - ASREPROasting.md
│   │   │   ├── H3 - LLMNR&NBT-NS Response Spoofing.md
│   │   │   └── H4 - Tomcat Manager Weak Credentials.md
│   │   ├── Logging output
│   │   ├── Misc files
│   │   └── Notes
│   │       ├── 10. AD Enumeration Research.md
│   │       ├── 11. Attack Path.md
│   │       ├── 12. Findings.md
│   │       ├── 1. Administrative Information.md
│   │       └── 2. Scoping Information.md
└──
```



Reminder: The folder and notetaking structure shown above is what has worked for us in our careers but will differ from person to person and engagement to engagement. We encourage you to try this out as a base, see how it works for you, and use it as a basis for coming up with a style that works for you. What's important is that we are thorough and organized, and there is no singular way to approach this. Obsidian is a great tool, and this format is clean, easy to follow, and easily reproducible from engagement to engagement. You could create a script to create the directory structure and the initial 10 Markdown files. You will get a chance to play around with this sample structure via GUI access to a Parrot VM at the end of this section.

## Formatting and Redaction

Credentials and Personal Identifiable Information (PII) should be redacted in screenshots and anything that would be morally objectionable, like graphic material or perhaps obscene comments and language. You may also consider the following:

- Adding annotations to the image like arrows or boxes to draw attention to the important items in the screenshot, particularly if a lot is happening in the image (don't do this in MS Word).
- Adding a minimal border around the image to make it stand out against the white background of the document.
- Cropping the image to only display the relevant information (e.g., instead of a full-screen capture, just to show a basic login form).
- Include the address bar in the browser or some other information indicating what URL or host you're connected to.

## Screenshots

Wherever possible, we should try to use terminal output over screenshots of the terminal. It is easier to redact, highlight the important parts (i.e., the command we ran in blue text and the part of the output we want to call attention to in red), typically looks neater in the document, and can avoid the document from becoming a massive, unwieldy file if we have loads of findings. We should be careful not to alter terminal output since we want to give an exact representation of the command we ran and the result. It is OK to shorten/cut out unnecessary output and mark the removed portion with `<SNIP>` but never alter output or add things that were not in the original command or output. Using text-based figures also makes it easier for the client to copy/paste to reproduce your results. It's also important that the source material that you're pasting *from* has all formatting stripped before going into your Word document. If you're pasting text that has embedded formatting, you may end up pasting non-UTF-8 encoded characters into your commands (usually alternate quotes or apostrophes), which may actually cause the command to not work correctly when the client tries to reproduce it.

One common way of redacting screenshots is through pixelation or blurring using a tool such as Greenshot. [Research](#) has shown that this method is not foolproof, and there's a high likelihood that the original data could be recovered by reversing the pixelation/blurring technique. This can be done with a tool such as [Unredacter](#). Instead, we should avoid this technique and use black bars (or another

solid shape) over the text we would like to redact. We should edit the image directly and not just apply a shape in MS Word, as someone with access to the document could easily delete this. As an aside, if you are writing a blog post or something published on the web with redacted sensitive data, do not rely on HTML/CSS styling to attempt to obscure the text (i.e., black text with a black background) as this can easily be viewed by highlighting the text or editing the page source temporarily. When in doubt, use console output but if you must use a terminal screenshot, then make sure you are appropriately redacting information. Below are examples of the two techniques:

### Blurring Password Data

```
[htb-student@par01]~/Desktop/HTB_Academy/Inlanefreight Penetration Test
$crackmapexec smb 172.16.5.5 -u asmith -p [REDACTED]
SMB 172.16.5.5 445 DC01 [*] Windows 10.0 Build 17763 x64 (name:DC01) (domain:INLANEFREIGHT.LOCAL) (signing:True) (SMBv1:False)
SMB 172.16.5.5 445 DC01 [+] INLANEFREIGHT.LOCAL\asmith:[REDACTED]
[htb-student@par01]~/Desktop/HTB_Academy/Inlanefreight Penetration Test
$
```

### Blanking Out Password with Solid Shape

```
[htb-student@par01]~/Desktop/HTB_Academy/Inlanefreight Penetration Test
$crackmapexec smb 172.16.5.5 -u asmith -p [REDACTED]
SMB 172.16.5.5 445 DC01 [*] Windows 10.0 Build 17763 x64 (name:DC01) (domain:INLANEFREIGHT.LOCAL) (signing:True) (SMBv1:False)
SMB 172.16.5.5 445 DC01 [+] INLANEFREIGHT.LOCAL\asmith:[REDACTED]
[htb-student@par01]~/Desktop/HTB_Academy/Inlanefreight Penetration Test
$
```

Finally, here is a suggested way to present terminal evidence in a report document. Here we have preserved the original command and output but enhanced it to highlight both the command and the output of interest (successful authentication).

```
$ crackmapexec smb 172.16.5.5 -u asmith -p <PASSWORD REDACTED>
SMB 172.16.5.5 445 DC01 [*] Windows 10.0 Build 17763 x64 (name:DC01)
(domain:INLANEFREIGHT.LOCAL) (signing:True) (SMBv1:False)
SMB 172.16.5.5 445 DC01 [+] INLANEFREIGHT.LOCAL\asmith:<PASSWORD REDACTED>
```

The way we present evidence will differ from report to report. We may be in a situation where we cannot copy/paste console output, so we must rely on a screenshot. The tips here are intended to provide options for creating a neat but accurate report with all evidence represented adequately.

## Terminal

Typically the only thing that needs to be redacted from terminal output is credentials (whether in the command itself or the output of the command). This includes password hashes. For password hashes, you can usually just strip out the middle of them and leave the first and last 3 or 4 characters to show there was actually a hash there. For cleartext credentials or any other human-readable content that needs to be obfuscated, you can just replace it with a `<REDACTED>` or `<PASSWORD REDACTED>` placeholder, or similar.

You should also consider color-coded highlighting in your terminal output to highlight the command that was run and the interesting output from running that command. This enhances the reader's ability to identify the essential parts of the evidence and what to look for if they try to reproduce it on their own. If you're working on a complex web payload, it can be difficult to pick out the payload in a gigantic URL-encoded request wall of text if you don't do this for a living. We should take all opportunities to make the report clearer to our readers, who will often not have as deep an understanding of the environment (especially from the perspective of a penetration tester) as we do by the end of the assessment.

## What Not to Archive

When starting a penetration test, we are being trusted by our customers to enter their network and "do no harm" wherever possible. This means not bringing down any hosts or affecting the availability of applications or resources, not changing passwords (unless

explicitly permitted), making significant or difficult-to-reverse configuration changes, or viewing or removing certain types of data from the environment. This data may include unredacted PII, potentially criminal info, anything considered legally "discoverable," etc. For example, if you gain access to a network share with sensitive data, it's probably best to just screenshot the directory with the files in it rather than opening individual files and screenshotting the file contents. If the files are as sensitive as you think, they'll get the message and know what's in them based on the file name. Collecting actual PII and extracting it from the target environment may have significant compliance obligations for storing and processing that data like GDPR and the like and could open up a slew of issues for our company and us.

## Module Exercises

We have included a partially filled-out sample Obsidian notebook in the Parrot Linux host that can be spawned at the end of this section. You can access it with the credentials provided using the following command:

```

Notetaking & Organization

dbcypn@htb[/htb]$ xfreerdp /v:10.129.203.82 /u:htb-student /p:HTB@cademy_stdnt!
    
```

Once connected in, you can open Obsidian from the Desktop, browse the sample notebook, and review the information that has been pre-populated with some sample data based on the lab that we will work against later in this module when we work through some optional (but highly encouraged!) exercises. We also provided a copy of this Obsidian notebook which can be downloaded from [Resources](#) in the top right of any section in this module. Once downloaded and unzipped, you can open this in a local copy of Obsidian by selecting **Open folder as vault**. Detailed instructions for creating or opening a vault can be found [here](#).

## Onwards

Now that we've gotten a good handle on our notetaking and folder organization structure and what types of evidence to keep and not to keep, and what to log for our reports, let's talk through the various types of reports our clients may ask for depending on the engagement type.

### VPN Servers

**Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

### PROTOCOL

☒ UDP 1337
 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



### Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

### Pwnbox Location

AU

1262ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

## Questions

Answer the question(s) below to complete this Section and earn cubes!



Download VPN  
Connection File

Target: [Click here to spawn the target system!](#)

RDP to with user "**htb-student**" and password "**HTB\_@cademy\_stdnt!**"

+ 1

What tool mentioned in this section can make logging a session easier?

Submit your answer here...

Submit

+ 2

Steve is learning about the tool that can make logging a session easier. He messages you for help mentioning that he would like to try to split the panes vertically. What do you tell him? (Answer format: [key] + [key] + [key], i.e., fill in the values for "key" and leave the brackets and + signs.)

Submit your answer here...

Submit

Hint

## Optional Exercises

Challenge your understanding of the Module content and answer the optional question(s) below. These are considered supplementary content and are not required to complete the Module. You can reveal the answer at any time to check your work

answer at any time to check your work.

Target: [Click here to spawn the target system!](#)

Connect to the testing VM using Xfreerdp, play around with the assessment directory structure and Obsidian notebook, and experiment with Tmux logging. Type DONE as your answer when you have finished.

Submit your answer here...

Submit

Reveal Answer

Previous

Next

Resources

Go to Questions

## Table of Contents

### Intro

Introduction to Documentation & Reporting



### Preparation

Notetaking & Organization

Types of Reports

Components of a Report

### Reporting

How to Write Up a Finding

Reporting Tips and Tricks

### Next Steps

Documentation & Reporting Practice Lab

Beyond this Module - Documentation & Reporting

### My Workstation

OFFLINE

Start Instance

1 / 1 spawns left



